

9주차 과제

LoRA

1. Abstract

- 대형 언어모델(GPT-3 등)은 **fine-tuning** 시 모든 파라미터를 업데이트해야 하므로
 - 저장 비용
 - 연산 비용
 - 배포 비용이 매우 크다.
- 이를 해결하기 위해 **LoRA (Low-Rank Adaptation)** 제안
 - 기존 weight는 고정(freeze)
 - 대신 **low-rank 행렬 A, B만 학습**
- 결과:
 - 학습 파라미터 수 최대 **10,000배 감소**
 - GPU 메모리 사용 감소
 - 성능은 full fine-tuning과 동일하거나 더 우수

2. Introduction

기존 문제

- LLM은 downstream task마다 별도의 모델 필요
- GPT-3 (175B)의 경우:
 - task마다 모델 복제 → 현실적으로 불가능
- 기존 해결 방법:
 1. Adapter
 2. Prompt tuning→ 하지만
- 성능 저하 또는

- latency 증가 문제 존재

핵심 문제

- “모든 파라미터를 업데이트해야 하는가?”
-

3. Related Works

(1) Full Fine-tuning

- 모든 파라미터 업데이트
- 성능 좋지만 비용 매우 큼

(2) Adapter

- 중간에 작은 네트워크 삽입
- 문제:
 - inference latency 증가

(3) Prefix / Prompt tuning

- 입력 토큰에 학습 가능한 벡터 추가
- 문제:
 - sequence length 감소
 - optimization 어려움

(4) Low-rank 관련 연구

- 딥러닝 모델은 본질적으로 **low intrinsic dimension**을 가진다는 연구 존재
-

4. Method

핵심 아이디어

- weight 업데이트 ΔW 는 **low-rank 구조일 것**이라는 가설
-

LoRA 수식

기존:

$$h = Wxh = Wx$$

$$h = Wx$$

LoRA 적용:

$$h = W_0 x + \Delta W x = W_0 x + \Delta W x$$

$$h = W_0 x + \Delta W x$$

$$\Delta W = BA \quad \Delta W = BA$$

$$\Delta W = BA$$

full matrix 대신 rank r 분해로 표현

핵심 특징

1. W_0 는 고정 (freeze)
2. A, B만 학습
3. inference 시:
 - $W = W_0 + BA$ $W = W_0 + BA$ 로 합쳐서 사용
 - → latency 증가 없음

Transformer 적용

- 적용 위치:
 - attention layer의
 - W_q, W_k, W_v, W_o W_q, W_k, W_v, W_o
- 실험에서는 주로:
 - **W_q, W_v 만 적용** (가성비 최적)

5. Experiments

실험 대상

- RoBERTa
- DeBERTa
- GPT-2
- GPT-3 (175B)

주요 결과

(1) GLUE benchmark

- LoRA가 fine-tuning과 동일 또는 더 높은 성능

(2) GPT-2 (NLG)

- LoRA > Adapter / Prefix tuning

(3) GPT-3 결과

- LoRA가 모든 task에서 최고 성능
 - 파라미터 수:
 - FT: 175B
 - LoRA: ~4.7M
-

6. Discussion

주요 발견

(1) 매우 낮은 rank로도 충분

- $r = 1 \sim 4$ 에서도 성능 유지

→ weight update는 실제로 매우 low-rank

(2) ΔW 는 기존 W 와 완전히 다르지 않음

- 기존 weight의 중요한 방향을 **증폭(amplify)** 하는 역할

(3) 어떤 weight에 적용할지 중요

- $W_q + W_v$ 조합이 가장 효과적

한계

- 서로 다른 task를 동시에 batch 처리 어려움
 - 어떤 layer에 적용할지 heuristic 기반
 - 이론적 설명 아직 부족
 - LoRA
 - parameter-efficient
 - compute-efficient
 - latency-free
 - adaptation 방법
 - 핵심 가치:
 - 대형 LLM을 현실적으로 활용 가능하게 만든 방법
-

8. 기존 연구 대비 차별점

방법	특징	한계
Fine-tuning	최고 성능	비용 매우 큼
Adapter	적은 파라미터	latency 증가
Prompt tuning	매우 효율적	성능 불안정
LoRA	low-rank update	구조 단순 + latency 없음

9. 핵심 아이디어 정리

- weight 업데이트는 full-rank가 필요 없다
- ΔW 는 low-rank로 충분히 표현 가능
- pretrained model은 그대로 두고
→ 작은 모듈만 학습하면 된다

10. 주요 수식 & 코드 관점 해석

핵심 수식

$$h = W_0 x + B A x$$

```
# 기존 linear layer
output = W @ x

# LoRA 적용
output = W0 @ x + (B @ A) @ x

# 효율적
output = W0 @ x + B @ (A @ x)
```

파라미터 비교

- 기존: $d \times k \times k$
- LoRA: $d \times r + r \times k \times r + r \times k \times r$
→ r 이 작으면 엄청 줄어듦

11. 공부 포인트

1. 왜 low-rank가 가능한가?

- 모델이 overparameterized
 - 실제 학습은 low-dimensional subspace에서 일어남
2. LoRA를 어디에 적용할까?
- attention weight가 핵심
3. 왜 성능이 유지되나?
- ΔW 는 기존 W 를 "보정"하는 역할
4. 실무 관점 핵심
- GPU 적게 써도 fine-tuning 가능
 - 여러 task를 하나의 모델로 관리 가능

RAG

1. 어떤 문제를 해결하려 했는가?

- 기존 대형 언어모델(LLM)은 **파라미터 내부에 지식을 저장**하지만
 - 최신 정보 업데이트가 어렵고
 - 근거(provenance) 제공이 어렵고
 - hallucination 문제가 존재함
 - 기존 retrieval 기반 모델은 대부분 **extractive QA에만 제한**
- 따라서 본 논문은

parametric memory(모델 내부) + non-parametric memory(외부 문서)를 결합한

RAG(Retrieval-Augmented Generation)을 제안

- 다양한 knowledge-intensive NLP task에서
더 정확하고, 근거 기반이며, 업데이트 가능한 생성 모델을 목표로 함

2. 연구 동기와 문제점

기존 LLM의 한계:

1. 지식 업데이트 불가

- 새로운 정보 반영하려면 재학습 필요

2. 설명 가능성 부족

- 어떤 근거로 답했는지 알기 어려움

3. hallucination

- 사실과 다른 내용을 생성

4. 지식 활용 한계

- 정확한 정보 retrieval 없이 내부 기억에만 의존

→ 해결 방향:

- 외부 지식(예: Wikipedia)을 활용하는

hybrid memory 구조 필요

3. 관련 연구 동향

1) Parametric-only 모델

- GPT, BART, T5
- 장점: end-to-end 학습, 범용성
- 단점: knowledge 업데이트 어려움

2) Retrieval 기반 모델

- REALM, ORQA
- retrieval + QA 구조
- 대부분 **extractive 방식**

3) Memory 기반 모델

- Memory Networks
- 외부 memory 사용

핵심 차별점

- 기존: 특정 task 전용
 - RAG:
범용 seq2seq 모델 + retrieval 결합
-

4. 연구 접근법과 모델 구조

전체 구조

RAG = **Retriever (DPR)** + **Generator (BART)**

(1) Retriever

- Dense Passage Retriever (DPR)
- query $x \rightarrow$ 관련 문서 z retrieval

수식:

$$p_{\eta}(z | x) \propto \exp(d(z)Tq(x))$$

- $q(x)$: query embedding
- $d(z)$: document embedding

→ MIPS (Maximum Inner Product Search) 사용

(2) Generator

- BART (seq2seq transformer)
- 입력: (x + retrieved document z)
- 출력: y 생성

(3) 핵심 아이디어: Latent variable z

- retrieved document z 를 **latent variable**로 취급
- 여러 문서를 marginalize

(4) 두 가지 RAG 구조

1) RAG-Sequence

- 하나의 document로 전체 sequence 생성

2) RAG-Token

- token마다 다른 document 사용

(5) 학습 방식

- end-to-end 학습
- retrieval supervision 없음

5. 실험 구성과 결과

데이터

- Wikipedia (21M passages)

- QA:
 - Natural Questions
 - TriviaQA
 - WebQuestions
 - Generation:
 - MS-MARCO
 - Jeopardy
 - Fact verification:
 - FEVER
-

주요 결과

1) Open-domain QA

- SOTA 달성
- 기존 DPR, REALM보다 성능 우수

2) Generation task

- BART 대비:
 - 더 factual
 - 더 specific
 - 더 diverse

3) Fact Verification

- retrieval supervision 없이도 높은 성능
-

6. 발견 및 한계

발견

1. retrieval + generation 결합이 효과적
 2. hallucination 감소
 3. 다양한 task에서 범용적으로 적용 가능
 4. 외부 memory 교체만으로 knowledge 업데이트 가능
-

한계

1. retrieval quality에 의존
2. 계산 비용 증가
3. Wikipedia bias 문제
4. end-to-end joint pretraining 부족

7. 결론 (Conclusion)

- RAG는
parametric + non-parametric memory 결합 모델
- 다양한 NLP task에서 강력한 성능
- 특히:
 - factual accuracy 향상
 - interpretability 증가
 - knowledge update 용이

→ 향후 연구:

- retrieval + generator 공동 pretraining

8. 기존 연구 대비 차별점

구분	기존	RAG
memory	parametric only	hybrid
retrieval	일부 task 전용	범용
generation	제한적	자유 생성
업데이트	어려움	index 교체

9. 핵심 아이디어

1. LLM + 검색엔진 결합
2. retrieval 결과를 **latent variable**로 처리
3. 여러 문서 기반 **marginalization**
4. token-level retrieval (RAG-Token)

10. 주요 코드 분석

```
# retrieval
docs = retriever(query)

# generation
outputs = []
for doc in docs:
    y = generator(query + doc)
    outputs.append(y)

# marginalization
final_output = aggregate(outputs)
```

핵심 구현 포인트

1. FAISS 기반 index
2. top-k retrieval
3. seq2seq 입력 concat ($x + z$)
4. end-to-end fine-tuning

-
- RAG + Knowledge Graph